

Real-Time Systems - Laboratory Exercise 4 (week 5)

Lab Note: These tasks are to be conducted individually (but you should discuss with your friend, don't get lost alone). Although you don't need to submit a report for assessment as in Melbourne, you should make a **note document** and keep your solution/s **code (*.c file/s)**, so that you can review later when doing the final project, as well as prepare for the In-class Tests.

In the Note document, you should have the required diagrams, screen captures and text to answer the questions. Your note should also contain examples for different input and expected output to demonstrate the behaviour of your program meets the outlined objective. Finally, keep the source files (*.c files) together with the note.

2. Semaphores

Semaphores can be used instead of mutexes to enforce mutual exclusion. In QNX, unnamed semaphores can only be used between threads within the same process in a similar fashion to pthread_mutex_ functions. To use a semaphore primitive, the corresponding semaphore library functions are: sem_init, sem_wait, sem_post and sem_destroy. Get a start by reading the following link about sem_init() and click on the related reference links to the other sem_ functions.

http://www.qnx.com/developers/docs/7.0.0/#com.qnx.doc.neutrino.lib_ref/topic/s/sem_init.html

You might also find it helpful to start reading from page 26 onwards from the [Getting_Started_with_QNX_Neutrino.pdf](#) or reading the only help version of the PDFs http://www.qnx.com/developers/docs/7.0.0/index.html#com.qnx.doc.neutrino.sys_arch/topic/kernel_SYNCHRO.html

To use a Semaphore you will need to include the library: `#include <semaphore.h>`

TASK 2A: Open a new Momentics project and rewrite overlap2.c to use unnamed semaphores instead of mutexes. You will need to use the following library functions: sem_init(), sem_wait(), sem_post(), sem_destroy().

In your report please provide the code snippets showing how you have implemented the above semaphore functions.

3. Named Semaphores

Named Semaphores have a major advantage over unnamed semaphores, mutexes and reader/writer locks, as Named Semaphores can be used between processes and even between other QNX nodes on the same network. They do this by creating an entry into a nodes file system in a similar fashion to how mqueue's do, thus they can be accessed by other nodes via the qnet networking service, at a location like: /dev/sem/tmp/my_sem

To use named semaphores, the named-semaphore manager (procnto) must be running - procnto manages named semaphores directly and enables you to see the Semaphore in the file system, which can be accessed by other nodes. For example:
/dev/sem/tmp/MyNamedSemaphore

The semaphore object can be accessed over the network by navigating /net directory on any QNX node present in the network (as long as qnet services is running). To use a Named Semaphore you will also need to include the library: `#include <fcntl.h>`

TASK 3A: Open a new Momentics project and rewrite [overlap2.c](#) to use Named Semaphores. Use the library functions: `sem_open()`, `sem_wait()`, `sem_post()`, `sem_close()`, and `sem_unlink()`. Make sure to check the file system (`/dev/sem`) and located the named semaphore in a QNX terminal. See:

http://www.qnx.com/developers/docs/7.0.0/index.html#com.qnx.doc.neutrino.sys_arch/topic/kernel_Semaphores.html

In your report please provide the code snippets showing how you have implemented the above semaphore functions and how they differ from unnamed semaphores.

Note: Named Semaphores require more overhead than unnamed semaphores, so they should only be used if really needed. That is, if you need a mutex between 2 or more processes running on different nodes on the same network.

Note: A significant difference between a Semaphore (named or unnamed) and a mutex is that the semaphore is a counting object, whereas a mutex lock and unlock operations are used together within the same thread to protect the codes critical section. Additionally, another difference between a mutex and a semaphore is that a mutex maintains priority inheritance, which can be useful for preventing priority inversion.

4. Barriers

A barrier is a mechanism which allows a defined number of threads to be synchronized or rendezvous at a given point in each thread. First the barrier must be initialised by calling `pthread_barrier_init()` with the third parameter being the number of threads that will participate in the barrier.

Lookup barriers in the QNX documentation or see: http://www.qnx.com/developers/docs/7.0.0/index.html#com.qnx.doc.neutrino.sys_arch/topic/kernel_Barriers.html

Each participating thread, when it reaches the desired synchronization point in its code, will call `pthread_barrier_wait()`, which blocks the thread until all the threads have called `pthread_barrier_wait()`, and then all the threads will continue to resume execution.

TASK 4A: make a copy of the [barrier1.c](#) file from Canvas and compile it in a new QNX Momentics project. Make sure that you understand the program, especially the library functions used, by looking up the functions in the QNX Help. To demonstrate this to yourself, add another thread to the program that includes a 10 second sleep (i.e. `sleep(10)`) and ensure that it synchronizes with the other threads.

5. Condition Variables

These are a generalization on semaphores/mutexes (in that they can allow some arbitrary condition to be tested as part of safely accessing shared data).

The simplest use of condition variables is shown in the following example:

Thread 1	Thread 2
<pre>pthread_mutex_lock(&m); // test condition until true while (!my_condition) // if not true block till changed pthread_cond_wait(&c, &m); //do something here pthread_mutex_unlock(&m);</pre>	<pre>pthread_mutex_lock(&m); // change condition my_condition = TRUE; pthread_mutex_unlock(&m); // signal that it has changed pthread_cond_signal(&c);</pre>

Essentially Thread 1 wants to *do something* safely (i.e. while a mutex is locked) but must first wait until `my_condition` is TRUE (e.g. when set TRUE by Thread 2). Thread 1 first locks the mutex (`m`), then tests `my_condition`. If `my_condition` is not true then Thread 1 calls `pthread_cond_wait` with two parameters, a condition variable (`c`) and the mutex (`m`). `pthread_cond_wait` must only be called when the mutex is locked, as it unlocks the mutex,

then blocks until another thread does a `pthread_cond_signal` on the condition variable, then when unblocked it locks the mutex again. `pthread_cond_wait` must be called in a while loop, as `my_condition` must be retested (in case some other thread has changed it back to false, or another thread did the `pthread_cond_signal` or even a `pthread_cond_broadcast`). When `my_condition` is finally true Thread 1 can *do something* and then unlock the mutex.

Note that it is important that Thread 2 lock the mutex while changing `my_condition` to ensure safety (this is particularly important if there are multiple CPUs to ensure that `my_condition` is safely updated in shared memory before another thread can lock it).

TASK 5A: Make a copy of the `cp1.c` file from Canvas and compile it in a new QNX Momentics project. Make sure that you understand the program, especially the library functions used, by looking up the functions in the QNX Help:

http://www.qnx.com/developers/docs/7.0.0/index.html#com.qnx.doc.neutrino.sys_arch/topic/kernel_Condvars.html

Now that you understand how the basics of a conditional variable you can make a copy of the `cp2.c` file from Canvas which extends the `cp1.c` example to have simulated data being produced by a producer thread and being made available to the consumer thread.

The code in the `cp2.c` works well as long as your produce and consumer threads are running at the same (synchronized) speed. However, what happens if the writing and reading speed of each process are different? One solution would be to use mqueues to manage the synchronisation, however the overhead of the mqueues might be too much given that the threads are part of the same process. A better option might be to implement a ring buffer, which is also sometimes called a circular buffer. In simplistic terms this could be done using the following code:

```
// Global:
#define Buffer_size          10
#define Number_of_packets   15
char buffer[Buffer_size];

// write process:
int index = 0;                // write process index position
char data_value = 'A';
buffer[index] = data_value;   // to write to buffer
index = (++index) % Buffer_size; // update index pointer for circular buffer

// read process:
int index = 0;                // read process index position
char data_value;
data_value = buffer[index];   // to read from buffer
index = (++index) % Buffer_size; // update index pointer for circular buffer
```

TASK 5B: Write and test a program with a producer thread and a consumer thread and implement a ring buffer to buffer the data items being sent from the producer to the consumer. You can do this by stating with the code in `cp2.c`, or writing your own solution using semaphores. You may find it helpful to modify the struct containing the buffer to be something like this:

```
typedef struct
{
    char buffer[Buffer_size];    // Buffer
    int readfinished;           // Used to tell producer that consumer is finished reading
    pthread_mutex_t mutex;      // needs to be set to PTHREAD_MUTEX_INITIALIZER;
    pthread_cond_t condvar;     // needs to be set to PTHREAD_COND_INITIALIZER;
}app_data;
```

The buffer should be of size 10. To simulate an example where data is produced faster than it is consumed, the producer thread will produce 15 characters in total at a rate of 2 per second. Hence the producer will write 2 characters to the buffer and then sleep for 1 second. The consumer thread, on the other hand, will only read and display 1 character per second from the buffer. You can use `sleep(1)` statements in each loop to control the timing of the loop. The data being sent is of no consequence so it is suggested that you start sending the character 'A' and just increment the data value by 1 after each character is sent so that the data changes value from 'A' to 'O'. As an example, the console output should look similar to this:

```
Starting consumer/producer (main thread priority:10)...
->Producer thread started (thread priority:10)...
producer: Data(A) from simulated h/w ready
producer: Data(B) from simulated h/w ready
->Consumer thread started (thread priority:10)...
consumer: got data from producer : A
producer: Data(C) from simulated h/w ready
producer: Data(D) from simulated h/w ready
consumer: got data from producer : B
producer: Data(E) from simulated h/w ready
producer: Data(F) from simulated h/w ready
consumer: got data from producer : C
producer: Data(G) from simulated h/w ready
producer: Data(H) from simulated h/w ready
consumer: got data from producer : D
producer: Data(I) from simulated h/w ready
producer: Data(J) from simulated h/w ready
consumer: got data from producer : E
producer: Data(K) from simulated h/w ready
producer: Data(L) from simulated h/w ready
consumer: got data from producer : F
producer: Data(M) from simulated h/w ready
producer: Data(N) from simulated h/w ready
consumer: got data from producer : G
producer: Data(O) from simulated h/w ready
consumer: got data from producer : H
->Producer has sent all items - now waiting for consumer to finish...
consumer: got data from producer : I
consumer: got data from producer : J
consumer: got data from producer : K
consumer: got data from producer : L
consumer: got data from producer : M
consumer: got data from producer : N
consumer: got data from producer : O
readfnnished = 1
->Consumer thread finished...
->Producer thread finished...

Main Terminating....
```

As you can see from this output, the consumer thread clearly lags behind the producer thread, but is able to receive all the 15 characters in order, even though the buffer can only hold 10 characters.

To gain full marks, you need to schedule the producer thread to have a thread priority of 5 and the consumer thread to a priority of 2 (the main thread will have a default priority of 10). Additionally, both threads must join in the main – hence the potential need for a *readfnnished* flag. Please place screen captures of your code running in your report as well as answers to the following questions:

- If no buffer limit checking is implemented, what is the maximum number of characters that can be sent before the buffer starts to corrupt itself given the 2 to 1 write/read ratio and buffer size of 10 ? _____

- If you based your code on [cp2.c](#), which used a mutex and condvar to provide synchronisation, why does the producer thread need to stay alive until the consumer thread is finished reading all 15 characters? Alternatively, you can discuss how you modified the code to enable the producer thread to terminate before the consumer finished reading the buffer? _____
- Why might it be safer for the producer thread to have the same priority or a higher priority than the consumer thread? _____

HINT

The code in these example programs does not check the return value from most of the function calls in order to show only the essential code. The return value is usually set to -1 if there was an error, and should be checked when safety and reliability is important.

Also note that all the mechanisms covered in this lab are not all POSIX function compliant. Check at the end of the help on each function to determine whether the function is POSIX compliant or Neutrino (QNX) specific.

Also note that while most of the functions which start with pthread_ are POSIX compliant, a few are not (e.g. sleep-on-locks).